

Basic (your first FastAPI route) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper FastAPI), **Advanced** (how a real service's health check actually works). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A FastAPI route is a normal Python function with a decorator on top that tells FastAPI which URL and HTTP method should run it. `GET /health` means: when a client visits `/health` with a GET request, run this function and send back whatever it returns as the reply.

Things to use:

- `from fastapi import FastAPI`
- `app = FastAPI()` — one app object for your whole service.
- `@app.get("/health")` — the decorator, placed directly above your function.
- `return {"status": "ok"}` — a plain dict; FastAPI turns it into JSON for you.
- Run it with `uvicorn main:app --reload`, then hit it with `curl localhost:8000/health`.

Intermediate Version

`uvicorn main:app` is worth reading literally: `main` is the filename (`main.py`, no `.py`), `app` is the variable name you assigned `FastAPI()` to. Uvicorn imports that module and looks for that variable. `--reload` restarts the server automatically every time you save the file — useful while developing, never used in production.

The `-> dict` return type hint doesn't validate anything at runtime the way a Pydantic model does (more on that below), but it documents intent and shows up in FastAPI's automatically generated docs.

The exact pieces:

- `from fastapi import FastAPI` and `app = FastAPI()` at module level — created once, reused by every route.
- `@app.get("/health")` registers the function that follows into FastAPI's internal route table; the decorator itself does nothing until a matching request arrives.
- `def health() -> dict:` — the type hint is documentation, not a runtime check.
- Visit `http://localhost:8000/docs` after starting the server — FastAPI builds an interactive Swagger UI from your routes automatically, no extra code needed.

Advanced Version

A health check that always returns `{"status": "ok"}` no matter what can never actually catch anything wrong — it just proves the process can respond to HTTP at all. That's still useful (it's called a **liveness** check), but it's worth knowing the difference from a **readiness** check, which asks a harder question: "can this service actually do its job right now?" (e.g., can it reach its database?).

Why the distinction matters in real deployments: if a liveness check fails, the usual fix is to restart the process. If a readiness check fails, restarting won't help — the database being down isn't fixed by restarting your API — so the service should stop receiving new traffic without being killed and restarted in a loop. This document only asks for one `/health` route, but knowing this distinction is why some real APIs have both `/health` (or `/live`) and `/ready`.

The other real upgrade: a typed response via Pydantic, not a bare dict, so the reply shape is enforced and shows correctly in the OpenAPI schema — the same pattern `chat_route` builds on next.

Pieces:

- `from pydantic import BaseModel` then `class HealthResponse(BaseModel): status: str`.
- `@app.get("/health", response_model=HealthResponse)` — FastAPI validates your *return value* against this model too, not just incoming requests.
- `from fastapi.testclient import TestClient` — lets you call your routes directly in a test, with no server actually running: `client = TestClient(app)`, `response = client.get("/health")`, `assert response.status_code == 200`.

Difference between Basic, Intermediate, and Advanced: Basic names the plain idea and the tools for the smallest possible working route. Intermediate explains what those same tools are actually doing under the hood — the uvicorn import string, the auto-generated `/docs`. Advanced questions whether "always return ok" is even the right design, introduces the liveness/readiness distinction real services use, and upgrades the bare dict to a typed Pydantic response — the same upgrade every later exercise in this document builds on.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
create app = FastAPI()

route GET /health:
    return {"status": "ok"}

run: uvicorn main:app --reload
```

Here's the whole thing — try running it and hitting it yourself:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health() -> dict:
    return {"status": "ok"}
```

Expected output if you run `uvicorn main:app --reload` then `curl localhost:8000/health`:

```
{"status": "ok"}
```

Intermediate Version

```
define HealthResponse(BaseModel): status: str

create app = FastAPI()

route GET /health, response_model=HealthResponse:
    return HealthResponse(status="ok")
```

```
from fastapi import FastAPI
from pydantic import BaseModel
```

```

app = FastAPI()

class HealthResponse(BaseModel):
    status: str

@app.get("/health", response_model=HealthResponse)
def health() -> HealthResponse:
    return HealthResponse(status="ok")

```

Run it the same way as Basic, then check `http://localhost:8000/docs` — you should see `/health` listed with a documented `HealthResponse` schema, which the bare-dict version from Basic never showed.

Advanced Version

```

define HealthResponse(BaseModel): status: str

create app = FastAPI()

a module-level variable pretending to be a database connection flag

route GET /health:
    return HealthResponse(status="ok")    # pure liveness, no dependency check

route GET /ready:
    if the pretend database flag says "down":
        return a 503 with a clear reason
    else:
        return HealthResponse(status="ok")

```

Here's most of it — fill in the `/ready` check yourself:

```

from fastapi import FastAPI, Response
from pydantic import BaseModel

app = FastAPI()

class HealthResponse(BaseModel):
    status: str

# pretend this gets set by real startup/shutdown code elsewhere
database_is_reachable = True

@app.get("/health", response_model=HealthResponse)

```

```
def health() -> HealthResponse:
    return HealthResponse(status="ok")

@app.get("/ready", response_model=HealthResponse)
def ready(response: Response) -> HealthResponse:
    # your turn: if database_is_reachable is False, set response.status_code = 500
    # and return HealthResponse(status="not ready") instead
    ...
    return HealthResponse(status="ok")
```

Try setting `database_is_reachable = False` by hand and confirming `/ready` returns 503 while `/health` still returns 200 — that's the entire point of separating the two.

Difference between Basic, Intermediate, and Advanced: Basic returns a hardcoded dict with no validation on the way out. Intermediate wraps the same reply in a typed `HealthResponse` model, so FastAPI checks it and documents it automatically. Advanced adds a second route that can genuinely fail — proving the liveness/readiness split from Hint 1 with a real (if fake) dependency check, instead of two routes that both always say "ok."

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)